# Form Architecture & Runtime Schema Validation (React Hook Form + Zod)

## Assignment

### Problem Statement:

Build the **Job Application form** for a company careers portal, using React, TypeScript, React Hook Form and Zod.

A candidate applies for an open position by completing a five-step form:

| Step | Section | Contents |
|---|---|---|
| 1 | Personal Details | Full name, email, phone, date of birth, current city |
| 2 | Education | A repeating list of qualifications - degree, institution, year of passing, percentage. At least one entry is required. |
| 3 | Experience | The candidate selects Fresher or Experienced, and the fields shown change accordingly. A fresher gives availability to join. An experienced candidate gives total experience, notice period, and a repeating list of previous companies with role and from / to dates. |
| 4 | Application Details | Position applied for, expected joining date, portfolio link (optional), cover note (optional), acceptance of terms |
| 5 | Review & Submit | A read-only summary of everything entered, with a Submit button |

Decide the validation rules yourself - required fields, formats and sensible limits - including at least three rules that depend on more than one field. For example: the candidate must be 18 or older, a previous company's "to" date must be after its "from" date, and the expected joining date must be in the future.

Submission goes to a mock API function that you write. It should pause for about 1.5 seconds and then simulate three outcomes: a field-level validation failure returned by the server, a server error, and a success. The form must handle all three differently.

### What We Are Looking For:

This assignment is assessed on **how the form is structured**, not on how it looks. Styling earns no marks.

- **The schema is the single source of truth.** All validation rules live in Zod schemas and all TypeScript types are inferred with `z.infer`. No validation logic inside components, and no hand-written types duplicating a schema.
- **Fields are reusable components.** A small set of field components that register themselves through `useFormContext` and render their own label and error. No repeated label / input / error markup inside the step components.
- **One form instance for the whole wizard.** A single `useForm` with `FormProvider`, the current step validated with `trigger`, and data preserved when the user navigates back. Form values must not be copied into Redux, Zustand or Context.
- **Dynamic and conditional fields handled correctly.** `useFieldArray` for the repeating lists, keyed on `field.id`, with errors shown against the correct row. The fresher / experienced split modelled as a discriminated union.
- **The form is fast.** Narrow subscriptions - prefer `useWatch` over `watch()`, and isolate watched values so one change does not re-render the whole step.

- **The form is accessible.** Labels linked to inputs, `aria-invalid` on invalid fields, errors linked with `aria-describedby` and announced with `role="alert"`, focus moved to the first invalid field on failure, and the whole form completable using only the keyboard.

Use React Hook Form and Zod only - Formik, Yup and hand-rolled form state are not permitted. TypeScript must run in `strict` mode, and the application must build with no type errors and no console warnings.

## Deliverables:

- Upload the code to GitHub and **share the PR link and branch name**.
- A `README.md` covering: how to run the project, how you organized your schema files, which validation `mode` you chose and why, where form state lives and how the steps share it, two or three places where you deliberately avoided an unnecessary re-render, and anything left incomplete.
- **Screenshots or a short screen recording** of the complete flow, including the server validation error case and the server failure case.
- Share your **status updates over email**.